

VERIQO Readiness Tier Framework

Why this document exists

An earlier round's `docs/VERIQO_COMMERCIAL_READINESS_REPORT.md` closed with a single blanket verdict: "**PILOT-READY, NOT YET FULLY PRODUCTION-QUALIFIED.**" A subsequent, detailed technical review ("Tetapi ada satu masalah besar") correctly identified that verdict as imprecise: "pilot" collapses several commercially and technically distinct situations -- a design partner kicking the tires on synthetic data is not the same commitment as a paying customer trusting VERIQO with evidence that may end up in a dispute or a court filing, and neither is the same as a production-qualified deployment an enterprise customer's own security team has signed off on. The review's own critique, verbatim in substance: *the Commercial API Store is explicitly documented as in-memory-only; a blanket "PILOT-READY" claim sitting next to that fact is not honest labeling.*

This document replaces the blanket verdict with the review's own four-tier framework, gives each tier an objective, falsifiable entry bar, and states VERIQO's real position against each one -- citing the same kind of evidence (real code, real tests, real documents) every other report in this engagement cites. It also records what changed between the prior report and now: five of the review's named paid-pilot blockers (durable persistence, tenant cryptographic binding, evidence retention/legal hold, cryptographic signing, and an independent verifier that no longer permanently SKIPs signature checks) have since been closed. This document does not soften or walk back the prior gaps that remain -- it reports exactly which ones those are, same as before.

This document supersedes the "OVERALL" row of `VERIQO_COMMERCIAL_READINESS_REPORT.md` §1 and the "Bottom Line" of its §6; both are updated to point here rather than repeat the old single-phrase verdict (see that document's own revision note).

The Four Tiers

Each tier is a **commitment level**, not a code-quality score. A system can be excellent engineering and still not meet a higher tier's bar, because the bar is about what is at stake for whoever is using it, not how well the code that exists is written.

Tier 0 -- DEMO-READY

What it means: The system can be shown, live, to a prospect -- synthetic or scripted data, no external commitment, no real evidence at stake. The prospect is evaluating the idea and the user experience, not trusting the system with anything.

Entry bar: A real (not mocked, not slideware) end-to-end flow the prospect can watch or drive themselves, producing an artifact they can independently inspect.

Tier 1 -- DESIGN-PARTNER READY

What it means: A partner runs their own real workflow against the live system, with the shared understanding that the system is pre-commercial: active engineering support is expected, breaking changes are expected, and nothing durable or contractual is promised. Data used is real in shape but not yet relied upon for anything consequential (no claim is settled, no cargo is released, no legal position is taken on the strength of what the system outputs).

Entry bar: Everything Tier 0 requires, plus real authentication and authorization (not "anyone with the URL"), real per-partner data isolation, and an audit trail a partner could review during a working session.

Tier 2 -- PAID-PILOT READY

What it means: A customer is paying, and the evidence they submit is real evidence they intend to rely on -- to support a claim, a dispute position, or an internal decision. This is the tier the review's own priority-ranked list (durable persistence, tenant identity binding, evidence retention/legal hold, cryptographic signing/HSM, backup/restore, liveness/readiness, OIDC, external integrations, pen test, multi-region/72h soak) targets. Not every item on that list gates this tier -- the review's own reasoning is that a first paying customer asks "*is my evidence safe and not lost?*" before "*do you have distributed tracing?*" -- so this tier's bar is specifically the ones a rational first customer would refuse to pay without.

Entry bar (the review's own top five, verbatim in substance):

1. Durable persistence -- evidence and decisions survive a process restart or crash, with a proven recovery path.
2. Tenant identity binding -- the tenant a request acts as is derived from a verified identity, not a value the caller simply types into the request.
3. Evidence retention / legal hold -- evidence under a hold cannot be purged, and there is a real state machine governing what happens to evidence over time.
4. Cryptographic integrity -- evidence and dossiers are signed with a real, verifiable signature scheme, not merely hashed.
5. An independent verifier that does not silently assume authenticity -- it must either really check a signature or visibly say it cannot.

Items 6-10 on the review's list (liveness/readiness probes, OIDC, external network integrations, an independent pen test, multi-region/ 72-hour soak) are **not** gates for this tier -- they are named explicitly in §"Conditions Still Open" below as real, tracked work, consistent with the review's own instruction that they matter more at scale than at first-customer trust.

Tier 3 -- PRODUCTION QUALIFIED

What it means: The system has been qualified against real external infrastructure and independent scrutiny -- not merely built to be ready for it. This is the tier `docs/governance/ CANONICAL_READINESS_TAXONOMY.md`'s own `TEMPORARY_PRODUCTION_READINESS_CANDIDATE` ceiling structurally refuses to claim (that taxonomy has no `PRODUCTION_QUALIFIED` value at all -- see its own "final verdict" section). This document does not create one either.

Entry bar: A real HSM/cloud KMS in production custody, real multi-region infrastructure, a real independent penetration test report, a real 72-hour soak at production scale (not a synthetic harness proving the mechanism would work), a real independent security audit, and a live vulnerability feed integration -- each obtained through an actual external vendor or infrastructure engagement, not simulated internally. **READY_FOR_EXTERNAL_QUALIFICATION is not PRODUCTION_QUALIFIED** -- the review's own explicit instruction, and the exact distinction `CANONICAL_READINESS_TAXONOMY.md` already enforces structurally for every one of its 62 tracked gates.

VERIQO's Current Position

Tier	Verdict	Basis
Tier 0 -- DEMO-READY	YES	Unchanged from the prior report -- three real, independently-verifiable demo cases (<code>pkg/commercial/democases</code> , <code>docs/VERIQA_DEMO_CASES.md</code>), a real Evidence Dossier (<code>pkg/commercial/dossier</code>), and a real standalone verifier CLI (<code>cmd/veriqo-commercial-verify</code>).
Tier 1 -- DESIGN-PARTNER READY	YES	Real JWT/RBAC/API-key/audit stack (<code>pkg/platform/security</code>), real per-tenant data isolation proven adversarially (<code>TestTenantAIsolationFromTenantB</code> , <code>TestCommercialV1RoutesTenantIsolationOverHTTP</code>), and now (this round) a durable audit trail (<code>pkg/commercial/api/durable.go</code>) a design partner's own review session can trust survives between sessions.
Tier 2 -- PAID-PILOT READY	CONDITIONALLY READY -- upgraded from the prior report's implicit "not yet" (its own §2 flagged Tenant Isolation and Deployment YELLOW and named the Store in-memory-only). All five of the tier's gating items are now closed; the remaining conditions are named exhaustively below and are operational/documentation work, not further architecture.	See "What Closed This Round" and "Conditions Still Open" below.
Tier 3 -- PRODUCTION QUALIFIED	NOT YET	Unchanged. All 8 tracked blockers (<code>docs/governance/production-blockers.json</code>) read <code>READY_FOR_REAL_QUALIFICATION</code> -- the mechanism is proven, the real external qualification has not happened. Per the review's own instruction, this is reported honestly, not implied otherwise.

What Closed This Round (Tier 2's five gating items)

#	Item	Prior state	Now
1	Durable persistence	<code>pkg/commercial/api.Store</code> was documented in-memory-only; every case/evidence record was lost on restart.	CLOSED. <code>NewDurableStore</code> (<code>pkg/commercial/api/durable.go</code>) backs every mutating call with a real write-ahead log (<code>pkg/s</code> classified recovery), replaying identical state after a restart. <code>Backup/RestoreStoreFromBackup</code> implement and prove a genuine backup/
2	Tenant identity binding	<code>commercial_v1_routes.go</code> 's own doc comment named the exact gap: <i>"TenantID is currently a caller-supplied field ... not yet cryptographically bound to the verified JWT identity."</i>	CLOSED. <code>pkg/commercial/tenancy.Membership</code> grants Subject->Tenant authorization; <code>effectiveTenantID</code> (<code>veriqo/gateway/rest/commercial_v1_routes.go</code>) derives the tenant a request may act as from the verified JWT subject when J tenant the subject's Membership does not cover -- even a perfectly valid JWT naming a different tenant cannot act as one it is not granted. Falls b only when no authenticated identity exists at all (an unauthenticated deployment has no identity to bind to in the first place).
3	Evidence retention / legal hold	<code>pkg/governance/data</code> 's retention engine and <code>pkg/insurance/preservation</code> 's legal-hold mechanism existed but were not connected to <code>pkg/commercial/api.Store</code> -- a Commercial API caller had no way to place a hold or trigger retention on their own submitted evidence.	CLOSED , exactly per the review's own instruction not to redesign the existing mechanism, only integrate it. <code>SubmitEvidence</code> now places e moment it is finalized (required, not best-effort -- a failure here fails the submission). <code>SetRetentionPolicy/PlaceLegalHold/ReleaseLegalHold/EvaluateRetention/PurgeEvidence/Preservatio</code> (<code>pkg/commercial/api/preservation.go</code>) expose the full lifecycle. Legal hold blocks purge both structurally (no state-graph edge fr an explicit check in <code>data.Engine.Purge</code> .
4	Cryptographic signing	Evidence and dossiers were hashed but never signed -- there was no signature scheme in the reference build at all.	CLOSED. <code>Store.EnableSigning</code> wires the already-built <code>pkg/platform/security/keys</code> HSM/KMS abstraction (real key lifecy >REVOKED, real Ed25519 signing/verification, revocation proven retroactive even for signatures made while the key was active) into evidence s hash-then-sign (never the reverse): a <code>PackageSignature</code> signs the already-computed <code>PackageHash</code> , and <code>VerifyPackageHash</code> zero signature can never be part of what it signs.
5	Independent verifier, no permanent SKIP on signature	The verifier's own honest design always reported <code>SKIP</code> on signature verification -- there was no signing scheme for it to check against.	CLOSED, with an important nuance the review specifically asked to be named rather than glossed over. The verifier (<code>pkg/commercial-commercial-verify</code> -trusted-keys) now performs real Ed25519 signature and key-state verification, plus lineage cross-referencing a the caller supplies a TrustedKeyRegistry obtained through a channel outside the package itself (a published registry, a pilot's own key-di still honestly <code>SKIP</code> -- never a false <code>PASS</code> -- when no such registry is supplied, which remains this reference build's honest default and is what h /v1/packages/verify HTTP convenience route today (see that handler's own doc comment). This is now a key-distribution / PKI-opera not a missing capability in the verifier itself.

Two smaller items from the review's #6-#7 (liveness/readiness probes, production audit durability) were also closed this round even though they do not gate Tier 2: `GET /livez/GET /readyz` (`veriqo/gateway/rest/server.go`) give a real, dependency-aware readiness probe wired to `Store.Healthy()`; production audit durability is substantively subsumed by item 1 above, since the WAL now backs the whole Commercial Store including audit-ledger replay. An unbounded- request-body-size hardening gap (item #8) was also closed -- `http.MaxBytesReader` now bounds every Commercial API v1 POST route.

Conditions Still Open for Tier 2 (PAID-PILOT READY, unconditional)

None of the following require further architecture -- each is independently completable without redesigning anything this or prior rounds built:

Condition	Detail
Retention/signing capabilities not yet exposed over HTTP	<code>SetRetentionPolicy</code> , <code>PlaceLegalHold</code> , <code>ReleaseLegalHold</code> , <code>PurgeEvidence</code> , <code>EvaluateRetention</code> , <code>EnableSigning</code> are reachable today only via direct Go <code>Store</code> method calls -- there is no <code>/v1/...</code> route for a pilot customer's own tooling to call any of them. The capability exists and is tested; the HTTP surface for it does not yet.
POST <code>/v1/packages/verify</code> always verifies unsigned	The HTTP convenience route for package verification always passes a <code>nil</code> trusted-key registry (see item 5 above) -- there is no request shape on that route yet to accept a caller-supplied registry. The standalone CLI (the actual verifier of record) already supports <code>-trusted-keys</code> fully; this is a gap in the HTTP convenience wrapper specifically.
Key custody is reference-grade, not HSM-grade	Signing (item 4 above) uses <code>keys.MemoryKeyProvider/FileKeyProvider</code> -- real Ed25519 cryptography with a real key lifecycle, but key material lives in process memory or a local file, not a production HSM/cloud KMS. <code>pkg/platform/security/keys/awskms</code> defines the adapter shape for a future cloud KMS;

Condition	Detail
	it is not wired into a live AWS KMS tenancy (blocked on procurement -- see <code>CANONICAL_READINESS_TAXONOMY.md</code> 's <code>hsm_kms</code> blocker, <code>BLOCKED_EXTERNAL</code>). Acceptable for a pilot's trust model (the signature scheme and verification logic are real); not acceptable as permanent production key custody.
OIDC	Still a gap, unchanged from the prior report. <code>security.JWTMiddleware</code> verifies locally-signed HS256 tokens; a pilot customer wanting SSO via their own IdP needs a second, additional verification path built. Not a redesign of the existing JWT/tenant-binding work.
Incident response procedure	No written document exists. Organizational/process artifact, not code.
SLA / SLO draft	No document exists.
Backup/restore is code-proven, not operationally drilled	<code>TestStoreBackupAndRestoreRoundTrip</code> proves the mechanism round-trips correctly; no real deployment has run an actual backup-then-restore drill against a live customer-facing instance with a written runbook.
Independent security review / pen test	Not a strict code blocker for Tier 2 by this framework's own reasoning (a first customer asks about data safety before independent audits), but standard industry practice recommends at least a lightweight external review before any paying customer's real evidence is at stake. Full pen test remains a Tier 3 item (<code>pentest</code> blocker, <code>READY_FOR_REAL_QUALIFICATION</code>).

Independent Verifier Marketing Language (the review's item 2)

The review's own instruction: do not market the independent verifier as *"fully cryptographically independent"* while signature verification is permanently `SKIP`. That premise has changed -- signature verification is no longer permanently `SKIP`; it is real when a trusted key registry is supplied. The corrected, still-honest language this engagement uses going forward:

VERIQO's independent verifier performs real, structural, and cryptographic checks -- package hash, manifest integrity, custody chain, Merkle proof, lineage, and replay are always independently checked. **Signature and key-state verification are cryptographically real** when the verifying party supplies a trusted key registry obtained through their own channel (the point of "verify it yourself," not "trust our server's word for who signed it"). **Absent a supplied registry, those two checks honestly report `SKIP`, never a false `PASS`.**

This is more precise than either the review's original caution ("independent structural/integrity verifier" until signing exists, implying no cryptographic capability at all) or the prior report's imprecise language -- because neither one is accurate anymore. The capability is real and tested; whether a given verification run exercises it depends on an operational step (key distribution), not on missing code.

Relationship to the Canonical Readiness Taxonomy

`docs/governance/CANONICAL_READINESS_TAXONOMY.md` computes a different, lower-level axis: per-engineering-gate status (`VERIFIED_INTERNAL` / `READY_FOR_EXTERNAL_QUALIFICATION` / `BLOCKED_EXTERNAL` / `NOT_READY`), rolled up into a two-value verdict that structurally cannot express `PRODUCTION_QUALIFIED`. This document's four commercial tiers are a different axis -- **who can be let near the system, for what stakes** -- and do not conflict with that taxonomy: Tier 3 (`PRODUCTION_QUALIFIED`) is deliberately defined so that reaching it requires strictly more than that taxonomy's own ceiling (`TEMPORARY_PRODUCTION_READINESS_CANDIDATE`), never less. A reader should treat the two documents as complementary: the taxonomy answers "is each individual gate's engineering done," this document answers "given that, what is VERIQO actually allowed to be used for today."